Getting more from every experiment

Optimal design, the OWEA algorithm, and the `OWEAdesign` package

Min Yang

University of Illinois at Chicago

Two experiments

Experiment A	Experiment B
Same question. Same precision.	Same question. Same precision.
16 measurements. Cost: **$16**.	**4 measurements**. Cost: **$4**.

The only difference: how the measurements were arranged.

This talk is about how to find Experiment B—by hand for small problems, and with a click for everything else.

Roadmap

1. What optimal design does
2. A balance-scale story
3. The general set-up
4. The OWEA algorithm
5. The `OWEAdesign` package and web app
6. Take-home

What optimal design does

Statistics has two halves

Analysis (the famous half)

Given the data, what can we conclude? Estimates, confidence intervals, tests, models.

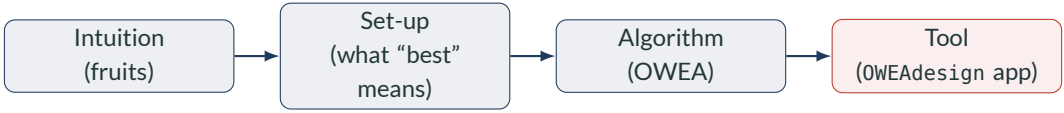
Design (the quiet half)

Before any data exist: *which* measurements should we take, and *how many* of each?

- Analysis can only squeeze out the information the data contain.
- **Design decides how much information there is to squeeze.**
- An **optimal design** gives the most precise answer for a fixed budget—or, equivalently, reaches a target precision with the smallest budget.

What we will do today

1. See the idea on a **balance scale** with four pieces of fruit (no formulas)
2. Write the same idea down in general (a few formulas, gently)
3. Meet **OWEA**, the algorithm that finds optimal designs (one picture)
4. Use the `OWEAdesign` **web app** to design a real experiment (no code)



A balance-scale story

The problem



We want the weights of four fruits: A, B, C, D.

- We have a balance and a set of standard weights.
- Every weighing has a small random error ϵ with variance σ^2.
- **Each weighing costs $1.** We can afford four.

How should we spend the four weighings?

An animated version of this example: `https://www.youtube.com/watch?v=5qewz8EYBzQ`

Design I: the obvious plan



Weigh each fruit on its own.

$$Y_1 = A + \epsilon_1, \qquad Y_2 = B + \epsilon_2,$$
$$Y_3 = C + \epsilon_3, \qquad Y_4 = D + \epsilon_4.$$

$$\hat{A} = Y_1, \quad \hat{B} = Y_2, \quad \hat{C} = Y_3, \quad \hat{D} = Y_4.$$

Precision

$$\mathrm{Var}(\hat{A}) = \mathrm{Var}(\hat{B}) = \mathrm{Var}(\hat{C}) = \mathrm{Var}(\hat{D}) = \sigma^2$$

Design II: weigh in pairs









Two fruits per weighing: both on the left pan, or one on each pan.

$$Y_1 = A + B + \epsilon_1, \qquad Y_2 = A - B + \epsilon_2,$$
$$Y_3 = C + D + \epsilon_3, \qquad Y_4 = C - D + \epsilon_4.$$

$$\hat{A} = \frac{Y_1+Y_2}{2},\ \hat{B} = \frac{Y_1-Y_2}{2},\ \hat{C} = \frac{Y_3+Y_4}{2},\ \hat{D} = \frac{Y_3-Y_4}{2}.$$

Precision

$\mathrm{Var}(\hat{A}) = \cdots = \mathrm{Var}(\hat{D}) = \sigma^2/2$

Same four weighings. Half the variance.

Design III: every fruit, every time






All four fruits on the scale every time: three on one pan, one on the other.

$$Y_1 = A + B + C - D + \epsilon_1, \quad Y_2 = A + B - C + D + \epsilon_2,$$
$$Y_3 = A - B + C + D + \epsilon_3, \quad Y_4 = -A + B + C + D + \epsilon_4.$$

$$\hat{A} = \frac{Y_1+Y_2+Y_3-Y_4}{4}, \quad \hat{B} = \frac{Y_1+Y_2-Y_3+Y_4}{4}, \quad \dots$$

Precision

$\mathrm{Var}(\hat{A}) = \cdots = \mathrm{Var}(\hat{D}) = \sigma^2/4$

Comparison

Same cost ($4)

Design	I	II	III
Variance	σ^2	$\sigma^2/2$	$\sigma^2/4$

Same precision ($\sigma^2/4$)

Design	I	II	III
Cost	**$16**	$8	**$4**

What the fruits teach us

- The *same* measurements, arranged differently, carry *four times* the information.
- Design III is the optimal design for this problem. It is also the one nobody would draw on a napkin.
- For four fruits a clever person finds Design III by hand. **For your experiment, who finds it?**

The general set-up

From fruits to formulas

Each weighing is a row of a matrix X with entries 1, 0 or -1 (left pan, absent, right pan):

$$Y = X\beta + \epsilon, \qquad \beta = (A, B, C, D)', \qquad \hat{\beta} = (X'X)^{-1}X'Y, \qquad \mathrm{Var}(\hat{\beta}) = \sigma^2 (X'X)^{-1}.$$

$$X_I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad X_{II} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}, \quad X_{III} = \begin{pmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{pmatrix}$$

$$(X_I'X_I)^{-1} = I, \qquad (X_{II}'X_{II})^{-1} = \tfrac{1}{2}I, \qquad (X_{III}'X_{III})^{-1} = \tfrac{1}{4}I.$$

The design problem in one sentence

Choose the rows of X (which measurements to take) so that $(X'X)^{-1}$ is as "small" as possible. $X'X$ is called the **information matrix**.

“Small” for a matrix: optimality criteria

A matrix is not a number, so we must say what we want to be small.

D-optimality

Minimise $\det(X'X)^{-1}$: the *volume* of the joint confidence region for all parameters. “Best overall precision.”

A-optimality

Minimise $\mathrm{tr}(X'X)^{-1}$: the *sum of the variances* of the estimates. “Best average precision.”

- Both belong to one family of criteria, Φ_p ($p = 0$ gives D, $p = 1$ gives A).
- Often we care about a **subset** of the parameters, or a function $g(\theta)$ of them (a slope, a target dose, a ratio). The criterion is then applied to that part only.
- For the fruits, every criterion agrees: Design III wins. In general they can disagree, so you must choose.

Designs as “where” and “how much”

A design lists the distinct measurement settings and the share of the budget each one gets:

$$\xi = \{(x_1, w_1),\ (x_2, w_2),\ \ldots,\ (x_k, w_k)\}, \qquad \sum_i w_i = 1.$$

- x_i: a **support point** (a dose, a temperature, a factor setting).
- w_i: its **weight**, the fraction of runs made there.
- Information adds up: $M(\xi) = \sum_i w_i\, I(x_i)$.

With n runs available, an **exact design** rounds nw_i to whole numbers.

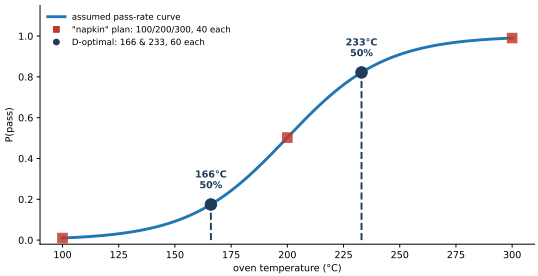
Example (coming up)

Oven temperature between 100 and 300 °C, 120 test pieces.

$\xi^* = \{(166°, 0.5),\ (233°, 0.5)\}$

$\Rightarrow$ 60 pieces at 166°, 60 at 233°.

Beyond straight lines: when the answer depends on the answer



Pass/fail data follow a logistic curve

$$P(\text{pass} \mid x) = \frac{e^{\alpha+\beta x}}{1 + e^{\alpha+\beta x}}.$$

- The information a run carries now depends on $\theta = (\alpha, \beta)$: a piece tested where the curve is flat tells you almost nothing.
- So the optimal design depends on the unknown θ: a **locally optimal design**, built around a best guess (pilot data, expert opinion).
- Afterwards we check: how good is the design if the truth is different?

The general problem, in one line

$$\xi^* = \arg\min_{\xi} \underbrace{\Phi_p}_{\text{criterion}} \Big(\underbrace{\Sigma\big(\xi, \overbrace{g(\theta)}^{\text{what we care about}}\big)}_{\text{variance of } \hat{g} \text{ under the model}} \Big), \qquad \xi = \{(x_i, w_i)\},\ x_i \in \underbrace{\mathcal{X}}_{\text{allowed settings}}$$

- **Criterion**: D, A, or any Φ_p.
- **Parameters of interest**: all of θ, a subset, or a function $g(\theta)$.
- **Model**: linear, logistic, Poisson, multinomial, ordinal, nonlinear regression, ...
- **Constraints**: the design region $\mathcal{X}$; possibly runs already made (multistage).

Two routes to a solution

Analytical: elegant theorems, but case by case (one model, one criterion at a time).
Algorithmic: one method for everything—if it is fast and reliable. That is the rest of this talk.

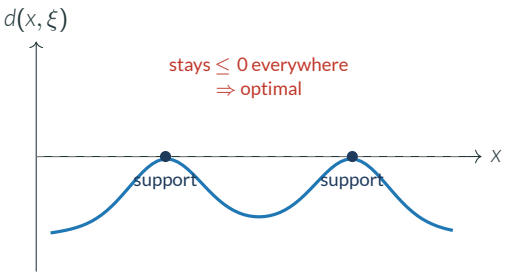
How do we know a design is optimal? One function to plot

For any candidate setting x, the **sensitivity function** $d(x, \xi)$ asks: *how much would the criterion improve if a little of the budget moved to x?*

General equivalence theorem (Kiefer)

ξ^* is optimal $\iff$ $d(x, \xi^*) \leq 0$ for every $x \in \mathcal{X}$, with equality at the support points of ξ^*.

- If $d(x, \xi) > 0$ somewhere, adding that x improves the design.
- For D-optimality: $d(x, \xi) = f(x)' M(\xi)^{-1} f(x) - k$.



The OWEA algorithm

What an algorithm must deliver

Wish list

- Any criterion: D, A, Φ_p, …
- Any parameters of interest: all, a subset, a function $g(\theta)$
- Any model: linear, GLM, nonlinear
- One-stage *and* multistage (add to an existing experiment)
- Fast enough for large candidate sets and many factors

The obstacle

Classical algorithms nudge the weights a little at a time, insisting that the criterion improves at every step ("monotone" updates). Safe, but slow—and the closed-form update rules only exist for special cases.

OWEA's idea: stop nudging. For a *given* set of support points, the optimal weights are the solution of a small system of equations—so *solve* it.

Yang, Biedermann & Tang (2013), *JASA*: "On optimal designs for nonlinear models: a general and efficient algorithm."

OWEA: two easy sub-problems, alternated

1. Given the points, find the weights

With support $S = \{x_1, \dots, x_k\}$ fixed, $\Phi_p(M(\xi))$ is a smooth function of the weights. Its minimum sits where

$$\frac{\partial \Phi_p}{\partial w_i} = 0 \quad \text{or} \quad w_i = 0.$$

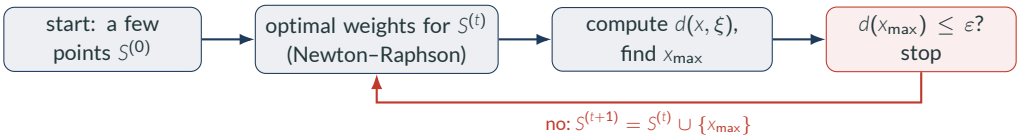
Newton–Raphson solves this in a handful of steps. Points whose weight hits 0 are dropped.

2. Given the weights, find the next point

Compute the sensitivity function $d(x, \xi)$ on the candidate grid $\mathcal{X}$.

$$x_{\max} = \arg\max_{x \in \mathcal{X}} d(x, \xi).$$

If $d(x_{\max}, \xi) \le \varepsilon$: **done, and certified optimal.** Otherwise **exchange**: add $x_{\max}$ to the support and go back to step 1.



Why it is fast, and why it is general

- **Exact weights every round.** The weights are solved for, not nudged, so there is no slow creep towards the optimum; points whose weight reaches zero are dropped at once, and the support stays small.
- **The criterion enters only through Φ_p and its derivative.** Swap D for A, or all parameters for a subset or a function $g(\theta)$: the loop is unchanged.
- **Multistage for free.** Runs already made contribute a fixed information matrix I_{ξ_0}; the algorithm optimises only what is added to it.
- **Any model.** All that is needed is the information a single run at x carries, $I(x, \theta)$.
- **A certificate at the end.** The stopping rule *is* the equivalence theorem, so the output comes with proof of optimality on the candidate set.

The `OWEAdesign` package and web app

`OWEAdesign`: the algorithm as a tool

R package

- `optimal_design()` – approximate designs (OWEA)
- `exact_design()` – whole-number allocations for n runs
- `verify_optimality()` – check any design
- `compound_design()` – one design serving several objectives
- `simulate_design()` – see the gain by simulation

```
remotes::install_github("minyangstat-eng/OWEAdesign")
```

Web app – no R, no installation

`https://statlab.math.uic.edu/OWEAdesign/`

A step-by-step wizard: describe the model, give a best guess, choose the criterion, click *Compute*. Verify, simulate, download.

- Models: linear, logistic, Poisson, multinomial, ordinal
- Continuous and discrete factors, several at once
- Multistage: start from a pilot design or data set

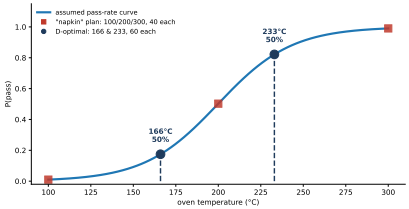
A real design problem: curing a coating

The experiment

- A protective coating on metal test pieces is **cured in an oven**; too cool and it stays soft. Each piece is scratch-tested and recorded as **pass or fail** (a binary response).
- The one factor we control: **oven temperature**, anywhere from 100 to 300 °C.
- Budget: **120 test pieces**, each cured at one temperature.

Goal

Estimate the whole pass-rate curve (α and β in the logistic model) precisely, so that a safe operating temperature can be chosen later.

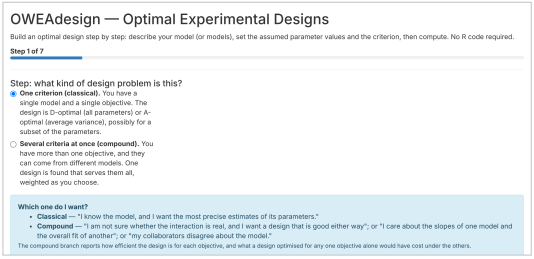


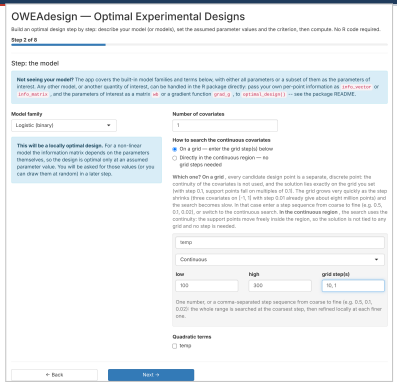
Prior knowledge, and the napkin plan

Bench experience: almost nothing passes at 100°, almost everything at 300°.

Natural plan: 100 / 200 / 300 °C, 40 pieces each. **Is that good?**

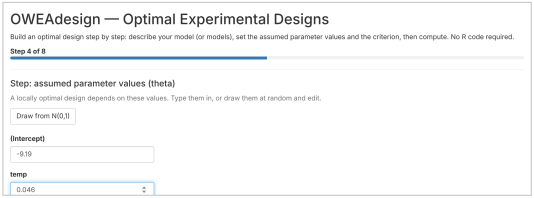
Step 1 – what kind of problem, and which model

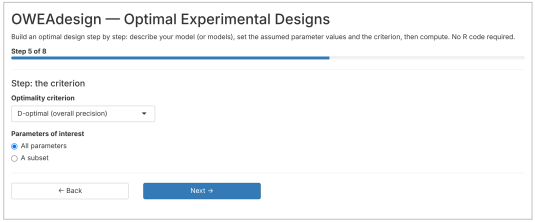




- One model, one goal ⇒ *classical*. (The *compound* branch handles several models at once.)
- Pass/fail ⇒ *Logistic (binary)*. One covariate, `temp`, from 100 to 300, searched on a coarse-then-fine grid (steps `10, 1`); a grid-free continuous search is the other option.

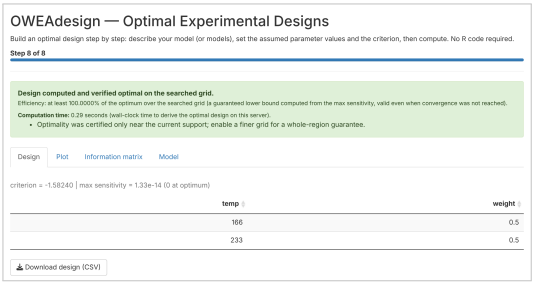
Step 2 – the best guess, and what “best” means





- The guess: intercept -9.19, slope 0.046 (pass rate $\approx$ 1% at 100°, $\approx$ 99% at 300°). With pilot data the app fits these for you.
- Criterion: D-optimal, all parameters – “the most precise picture of the whole curve.”

Step 3 – compute

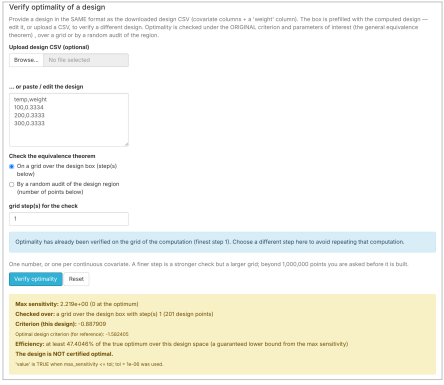




The answer

Two temperatures, **166° and 233°C**, half the pieces at each. Not low–medium–high, not the extremes: two points on the steep part of the curve, where each piece is most informative. Certified optimal (max sensitivity ≈ 0).

Step 4 – score the napkin plan



Paste any design into *Verify optimality*. For the napkin plan (100/200/300, equal weights):

$$\text{criterion (napkin)} = -0.888$$

$$\text{criterion (optimal)} = -1.582$$

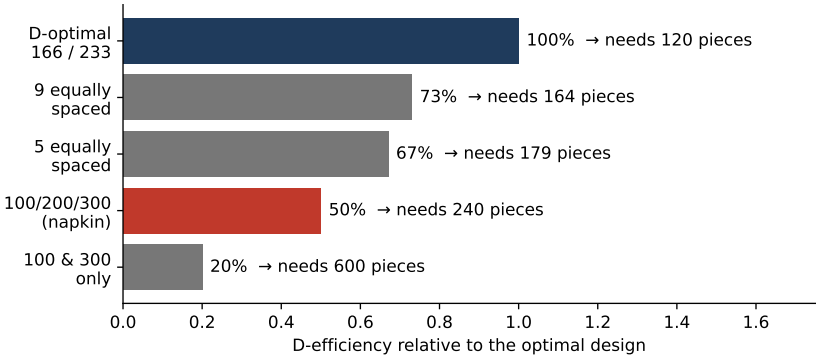
$$\text{gap} = 0.694 = \log 2$$

Meaning

The napkin plan is **50% efficient**: 120 pieces on it are worth 60 on the optimal design. To match the optimal design's precision it would need **240 pieces**. (The app also prints a guaranteed lower bound on the efficiency, here 47%.)

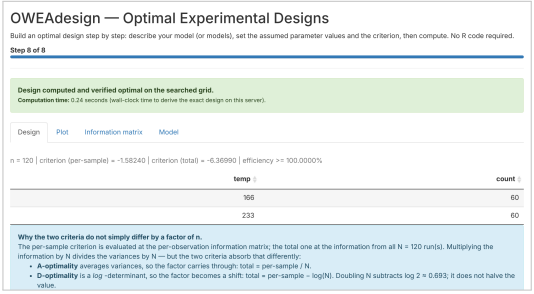
That is the fruit table again—in a real experiment.

How bad can a reasonable-looking plan be?



D-efficiency of common “sensible” allocations of 120 pieces, under the assumed curve. Spreading pieces across the flat parts of the curve is what costs information.

Step 5 – make it real: an exact design for 120 pieces



- Switch *Design type* to *Exact (integer runs)*, set $n = 120$, recompute.
- Result: **60 pieces at 166°, 60 at 233°.**
- Download as CSV and hand it to the lab.

Two flavours of the same design

Approximate: fractions of the budget (theory, verification).
Exact: whole runs (what you actually do).

Step 6 – don't trust a formula: simulate


Upload a custom design CSV (optional)

Browse... No file selected

... or paste a custom design

temp,count
100,40
200,40
300,40

Run custom design

parameter	Exact design (MSE)	Custom (MSE)	Custom / Exact design (MSE)	Exact design (median SE)	Custom (median SE)	Custom / Exact design (median SE)
(Intercept)	2.477448	434.032175	175.193268	1.042286	4.225477	4.054049
temp	0.000061	0.010852	177.986516	0.000021	0.000087	4.082656

The "/ Exact design" columns are ratios. Above 1 the computed design has the smaller error for that parameter (2.0 = twice the error); below 1 the other design does. The errors themselves are on each parameter's own scale, so only the ratio compares designs across a row.

Replications that converged: the exact design 1000/1000, custom 1000/1000.

The app simulates the experiment 1000 times from the assumed curve, fits the model each time and records the errors—for the optimal design and for any design you paste in.

Result

Median squared error, napkin ÷ optimal: $\approx 4\times$ for both parameters. The mean is far worse: with 40 pieces at 100°C every piece fails, and those runs carry no information at all.

Same 120 pieces, same cost, same model. Only the arrangement changed.

But my guess of the curve might be wrong

Change the guess and recompute

Suppose the truth is milder: 3% at 100°, 97% at 300°.

New optimal design: $\{155°, 244°\}$, equal weights.
The design moves a little, but keeps its shape: two points, equal weights.

Score the plan we already had

Verify $\{166°, 233°\}$ under the new truth: criterion gap 0.058 $\Rightarrow$ **94% efficient**.

The napkin plan never gets close.

- A locally optimal design is usually *robust* to moderate errors in the guess—and the app lets you check exactly how robust.
- If the *model itself* is in doubt, the *compound* branch finds one design that is efficient under several models at once — that is the next demo.
- Ran a pilot already? Paste the data: the app fits the curve and designs the *next* stage around what you have learned.

Demo 2 – what if the model itself is in doubt?

Two beliefs on the bench

- **Model 1: straight logistic.** Hotter is better, all the way to 300°.
- **Model 2: logistic with a quadratic term.** Over-curing hurts: the pass rate peaks around 230° and falls again at 300°.

Same experiment, same 120 pieces, same design region. Which design should we run?

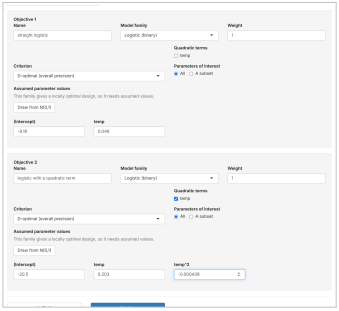
Each specialist is a gamble

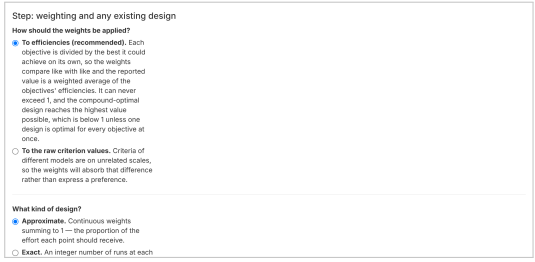
efficiency under →	Model 1	Model 2
design optimal for Model 1	1.00	**0.17***
design optimal for Model 2	**0.47**	1.00

*essentially useless: two support points cannot estimate three parameters.

A design that is perfect if you guessed the model right, and a disaster if you did not. **Compound design**: ask for one design that does well under *both*.

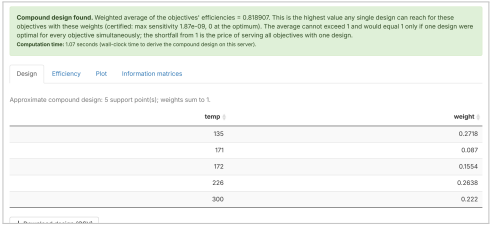
Demo 2 – setting it up in the app





- Choose *Several criteria at once (compound)*; the covariate and its range are shared by all objectives.
- Objective 1: logistic, *D*-optimal, the best guess from before. Objective 2: logistic with *Quadratic terms* ticked, *D*-optimal, its own guess. Weights 1 and 1, applied to *efficiencies*: care equally about each model's closeness to its own optimum.

Demo 2 – one design that serves both models

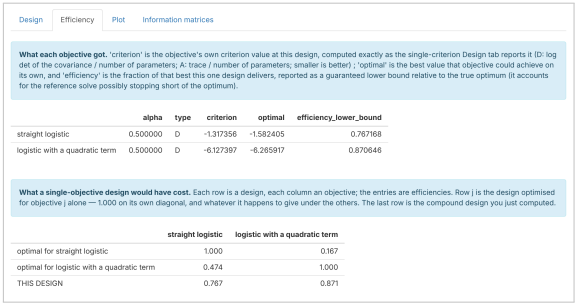


Compound design found. Weighted average of the objectives' efficiencies = 0.818907. This is the highest value any single design can reach for these objectives with these weights (certified: max sensitivity 1.83e−09, 0 at the optimum). The average cannot exceed 1 and would equal 1 only if one design were optimal for every objective simultaneously; the shortfall from 1 is the price of serving all objectives with one design.
Computation time: 1.07 seconds (wall-clock time to derive the compound design on this server).

Design | Efficiency | Plot | Information matrices

Approximate compound design: 5 support point(s); weights sum to 1.

temp	weight
135	0.2718
171	0.087
172	0.1554
226	0.2638
300	0.222



Design | Efficiency | Plot | Information matrices

What each objective got. 'criterion' is the objective's own criterion value at this design, computed exactly as the single-criterion Design tab reports it (D: log det of the covariance / number of parameters; A: trace / number of parameters; smaller is better) ; 'optimal' is the best value that objective could achieve on its own, and 'efficiency' is the fraction of that best this one design delivers, reported as a guaranteed lower bound relative to the true optimum (it accounts for the reference solve possibly stopping short of the optimum).

	alpha	type	criterion	optimal	efficiency_lower_bound
straight logistic	0.500000	D	-1.317356	-1.582405	0.767168
logistic with a quadratic term	0.500000	D	-6.127397	-6.265917	0.870646

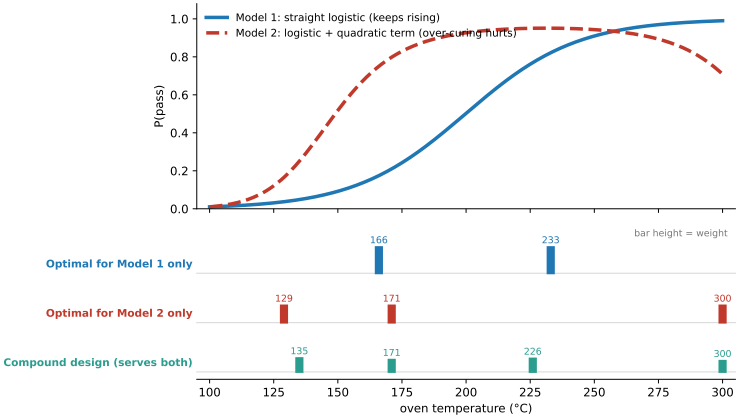
What a single-objective design would have cost. Each row is a design, each column an objective; the entries are efficiencies. Row j is the design optimised for objective j alone — 1.000 on its own diagonal, and whatever it happens to give under the others. The last row is the compound design you just computed.

	straight logistic	logistic with a quadratic term
optimal for straight logistic	1.000	0.167
optimal for logistic with a quadratic term	0.474	1.000
THIS DESIGN	0.767	0.871

Reading the result

Four temperatures (≈135, 171, 226, 300°C) with weights 0.27, 0.24, 0.26, 0.22. Under Model 1 it is **77%** efficient, under Model 2 **87%**; the average, 0.82, is certified to be the best any single design can do. That 18% shortfall is the price of insurance against picking the wrong model — versus losing 53% or 83% by betting on one specialist.

Demo 2 – the three designs side by side



The compound design keeps the two “steep-part” points of Model 1 and adds the end point and a low point that Model 2 needs to detect the bend.

Any design you prefer can be pasted into *Score another design* and rated on the same scale.

The same thing in R

```r
library(OWEAdesign)

# information vector f(x, theta) for a logistic model with one covariate
info_vec <- function(x, theta) {
  q <- c(1, x); eta <- sum(q * theta)
  (exp(eta / 2) / (1 + exp(eta))) * q
}

res <- optimal_design(info_vector   = info_vec,
                      theta         = c(-9.19, 0.046),   # best guess
                      design_box    = list(c(100, 300)), # allowed temperatures
                      step_sequence = c(10, 1),          # coarse -> fine grid
                      p             = 0)                 # 0 = D-optimal, 1 = A-optimal

cbind(temp = res$support, weight = res$weights)
#>   temp weight
#>    166    0.5
#>    233    0.5
```

Everything the app does is one function call away: `exact_design()`, `verify_optimality()`, `compound_design()`, `simulate_design()`.

Take-home

Take-home messages

1. **Design is where information is created.** The same budget, arranged well, can be worth two or four times as much.
2. **Optimal design has a precise meaning:** choose the settings and their shares so a chosen criterion of the information matrix is best—and one function, $d(x, \xi)$, certifies that you got there.
3. **OWEA** turns the theory into a general, fast procedure: solve for the weights, add the most valuable point, repeat.
4. **You do not need to be a statistician to use it.** Describe the experiment in the web app, click *Compute*, verify, simulate, download — and when the model is uncertain, ask for a compound design.

Next time you plan an experiment

Before you settle for low–medium–high, spend five minutes at `https://statlab.math.uic.edu/OWEAdesign/`.

Key reference and links

Key reference

Yang, M., Biedermann, S., and Tang, E. (2013). On optimal designs for nonlinear models: a general and efficient algorithm. *Journal of the American Statistical Association*, 108, 1411–1420.

R package

`https://github.com/minyangstat-eng/OWEAdesign`

```
remotes::install_github("minyangstat-eng/OWEAdesign")
```

Web app (no installation)

`https://statlab.math.uic.edu/OWEAdesign/`

Supported by the National Science Foundation.

Thank you.

Questions? myang2@uic.edu